
FACE Prep

BLOCK SWAP ALGORITHM

ARRAY DEFINITION

- Array Is Collection Of Elements Of Same Datatype Stored With Index Position
- Syntax
- Datatype Arrayname [Array Size]= {Elements};

Initialisation

Int Arr[5]= {10,29,35,77,88};

Char Arr[7]={ ‘A’, ‘C’, ‘G’, ‘P’, ‘H’, ‘I’ ,’K’};

BLOCK SWAP ALGORITHM

- **Block Swap Algorithm Or Array Rotation In Java**
- The Block Swap Algorithm For Array Rotation Is An Efficient Algorithm That Is Used For Array Rotation. It Can Do Your Work In O(n) Time Complexity.
- So, In Array Rotation, We Are Given An Array Arr[] Of Size N And A Number D That Define The No. Of The Element To Be Rotated.
- **Explanation** − On Rotation, We Will Shift The One Element To The Last Position And Shift The Next Elements To One Position.
- Element At Index 0 Will Be Shifted To Index N-1. And The Rest Of The Elements Are Shifted To The Previous Index.

Array Rotation Example

1. Input:

- Int Arr[] = {7, 9, 8, 0, 5, 1, 6, 4}, S = 8, D = 2
- **Output:** Result[] = {8, 0, 5, 1, 6, 4, 7, 9}
- **Explanation:** S Is The Size Of The Input Array. D = 2 Means We Have To Rotate The Array 2 Times. When We Rotate The First Time, We Get
- {9, 8, 0, 5, 1, 6, 4, 7}. Now, We Rotate The Array A Second Time, And We Get The Following.

2.Input:

- Int Arr[] = {6, 8, 7, 9, 0, 5, 1, 3, 2, 4}, S = 10, D = 5
- **Output:** Result[] = {5, 1, 2, 3, 4, 6, 8, 7, 9, 0}
- **Explanation:** After Rotating The Array 5 Times, We Get The Following:
- {5, 1, 2, 3, 4, 6, 8, 7, 9, 0}, Which Is Our Answer.

ALGORITHM STEPS

- Initialize A = arr[0..d-1] and B = arr[d..n-1]

1) Do following until size of A is equal to size of B

   a) If A is shorter, divide B into Bl and Br such that Br is of same length as A. Swap A and Br to change ABlBr into BrBlA. Now A is at its final place, so recur on pieces of B.

   b) If A is longer, divide A into Al and Ar such that Al is of same length as B Swap Al and B to change AlArB into BArAl. Now B is at its final place, so recur on pieces of A.

2) Finally when A and B are of equal size, block swap them

```
import java.io.*;

public class GFG {
    static void leftRotate(int arr[], int d, int n)
    {
        int i, j;
        if (d == 0 || d == n)
            return;
        /* If number of elements to be rotated is more than
         * array size*/
        if (d > n)
            d = d % n;
        i = d;
        j = n - d;
        while (i != j) {
            if (i < j) /*A is shorter*/
            {
                swap(arr, d - i, d + j - i, i);
                j -= i;
            }
            else /*B is shorter*/
            {
                swap(arr, d - i, d, j);
                i -= j;
            }
            // printArray(arr, 7);
        }
        /*Finally, block swap A and B*/
        swap(arr, d - i, d, i);
    }
```

```
/*UTILITY FUNCTIONS*/
    /* function to print an array */
    public static void printArray(int arr[], int size)
    {
        int i;
        for (i = 0; i < size; i++)
            System.out.print(arr[i] + " ");
        System.out.println();
    }

    /*This function swaps d elements
    starting at index fi with d elements
    starting at index si */
    public static void swap(int arr[], int fi, int si,
                            int d)
    {
        int i, temp;
        for (i = 0; i < d; i++) {
            temp = arr[fi + i];
            arr[fi + i] = arr[si + i];
            arr[si + i] = temp;
        }
    }

    // Driver Code
    public static void main(String[] args)
    {
        int arr[] = { 1, 2, 3, 4, 5, 6, 7 };
        leftRotate(arr, 2, 7);
        printArray(arr, 7);
    }
}
```

Output3 4 5 6 7 1 2

Time Complexity: O(n)
Auxiliary Space: O(1)

If we are using recursive function in program the space complexity is
AUXILIARY SPACE: O(LOG N)

THANK YOU

FACE Prep